Student 314551816 01.04.2026

Artificial Intelligence

Assignment 1 - Report

GithubLink: https://github.com/jspldrch/AI-Assignment1.git

This project implements a Human Activity Recognition system to classify four activities (sitting, walking, running, and climbing stairs) by using smartphone sensor data. By applying Random Forest, k-NN, and MLP models to features collected via the Phyphox app, the study evaluates classification performance and operational efficiency through five targeted experiments.

Dataset

This methodology describes how I created the dataset and developed the models for my project. I chose to focus on everyday activities including walking, running, sitting, and taking stairs because these are the most common movements people do in their daily lives. To make the data collection professional, I used the Phyphox app on my smartphone. I recorded each activity two times on different days and at different times. I did this because I wanted to have more variability in the data so the model does not just learn one specific movement from one specific moment. For the sensors, I tracked the gyroscope and the linear accelerometer. I specifically chose linear acceleration because it removes the gravity force, which allows the sensors to focus only on my actual body movement. During all recordings, I kept the mobile phone in my right front trousers pocket to keep the position the same.

To make the data better for the machine learning models, I did several preparation steps. First, I cleaned the raw files by removing the first and last fifteen seconds of every recording. This is important because it removes the noise created when I was putting the phone into my pocket or taking it out. Then I synchronized the sensors using a nearest-neighbor join at a frequency of fifty hertz. After the cleaning, I used a sliding window approach to create the actual dataset. I cut the data into segments of two seconds with a fifty percent overlap. This overlap is very useful because it ensures that no important movement pattern is lost at the edge of a window and it gives the models more examples to learn from. For every segment, I calculated thirty-six features like the mean, the standard deviation, and the root mean square. I also made sure to include variations in my movements, like changing the speed while I was walking or running and including both going up and down the stairs. This makes the dataset more difficult but also more realistic for the models. The final dataset is called final_features and in the data/processed folder on github.

Algorithm / Model

I selected three different models to compare how they perform on my sensor data. I chose these specific models because I wanted to see the difference between classic machine learning and deep learning.

The first model is the Random Forest. This is an ensemble method which means it uses many individual decision trees to make a final decision. I configured my forest to have one hundred trees. Each tree looks at the thirty-six features and makes its own prediction for the activity. After all trees are finished they take a majority vote to classify the activity. This model is very good for my data because it is strong against noise. For example if the phone moved slightly in my pocket during a sitting recording one tree might get confused but the other ninety-nine trees will still give the correct result. This makes the model very reliable and prevents it from overfitting to small mistakes in the data.

The second model is k-Nearest Neighbours. This model is very intuitive because it works based on distance in the feature space. I chose a value of five for the parameter k. This means when the model sees a new two-second window it looks for the five examples in the training data that are most similar to it. It then chooses the label that is most common among those five neighbours. This model works perfectly for my project because my PCA analysis showed that my four activities form very clear and separate groups. Since the distance between a sitting point and a running point is very large the model can easily find the right neighbours.

The third model is the Multi-Layer Perceptron which is the deep learning part of my assignment. This is a neural network that has an input layer two hidden layers and an output layer. My hidden layers have sixty-four and thirty-two neurons. This model is much more complex than the others because it uses layers of neurons to find hidden patterns. It can learn non-linear rules which means it can understand very complicated relationships between the gyroscope and acceleration sensors. For example it might find a specific combination of rotation and movement that only happens when I am going down the stairs. This makes it very powerful for recognizing difficult movements that a simple decision tree might miss.

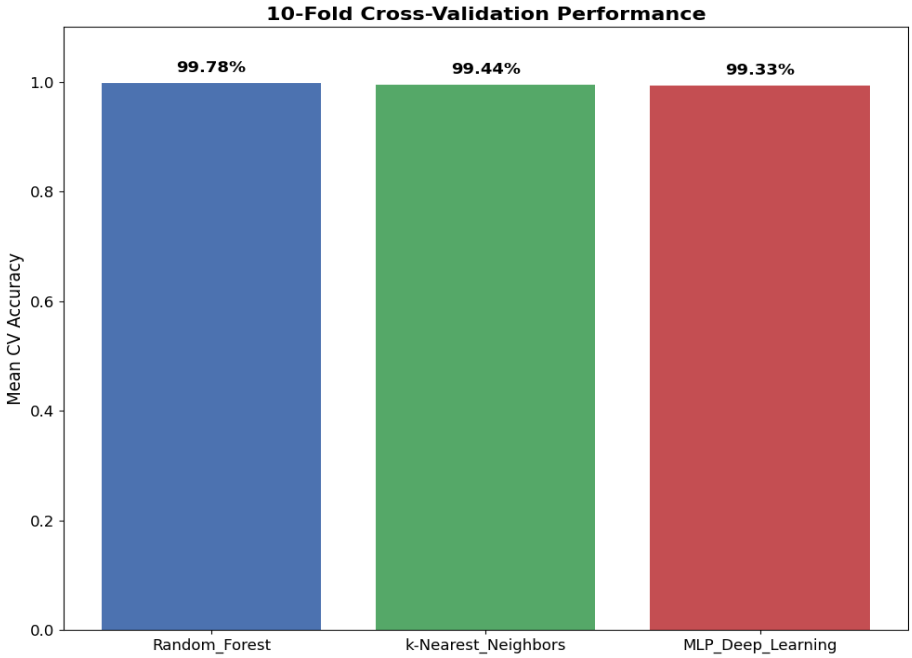
To make sure these models work correctly for all sensors I used a standard scaler to prepare the data. This is a very important step because the sensors have different units. The linear accelerometer measures in meters per second squared but the gyroscope measures in radians per second. Without scaling the models would think that the larger acceleration numbers are more important than the smaller gyroscope numbers. The standard scaler puts everything on

the same scale with a mean of zero and a standard deviation of one so the math works fairly for every feature.

Analysis

For the analysis of my supervised learning models I used several evaluation tools to check the quality of the classification. I did not just look at one simple test result but I used ten-fold stratified cross-validation to get the mean accuracy and the standard deviation for every model. I also created confusion matrices and analyzed the precision, the recall, and the F1-score for each of the four activities. These three metrics are very important because they show if a model is better at recognizing a specific movement like sitting compared to a more difficult one like taking stairs.

Figure 1: Results oft the 10-Fold Cross-Validation



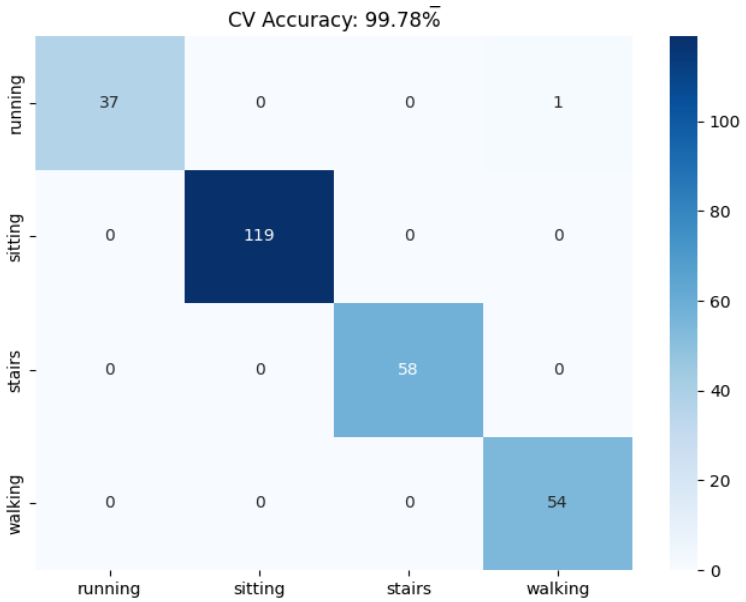
The first part of my analysis is the mean accuracy from the cross-validation which shows the overall performance. As pictured in *Figure 1*, the Random Forest achieved the highest mean accuracy of 99.78%. The k-Nearest Neighbors followed with an accuracy of 0.9944 and the Multi-Layer Perceptron reached 0.9933. These high numbers prove that my models are very stable and they work correctly on all parts of the data. Because the standard deviation is so small it means the models did not just get lucky with one part of the data but they truly understand the physical patterns of the different activities.

Table 1: Classification Report of Random Forest

	precision	recall	f1-score	support
running	1.0	0.974	0.987	38.0
sitting	1.0	1.0	1.0	119.0
stairs	1.0	1.0	1.0	58.0
walking	0.982	1.0	0.991	54.0
accuracy	0.996	0.996	0.996	0.996
macro avg	0.995	0.993	0.994	269.0
weighted avg	0.996	0.996	0.996	269.0

Table 1 shows that the precision for sitting, running and stairs of the random forest model are 1.00 which means the model never made a mistake for these classes. For walking the scores are slightly lower at around 0.98. This is because the confusion matrix (*Figure 2*) shows that one running sample was classified as walking. Conclusively, the recall is for all activities 1 besides for running (0.974) where one was misclassified. The F1-score combines precision and recall and is nearly 1.00 for every activity which shows a perfect balance.

Figure 2 Confusion Matrix of Random Forest

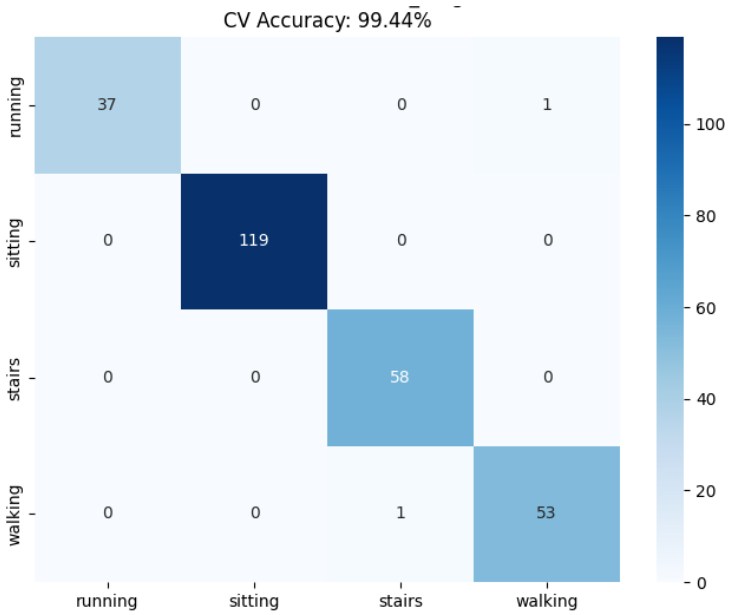


The k-Nearest Neighbours *(Table 2)* and the Multi-Layer Perceptron *(Table 3)* also have very high scores. For sitting they both achieved a precision, recall, and F1-score of 1.00 because the sensor data for sitting is very unique with almost zero acceleration. However, for running and walking they both had a small drop in the metrics. The confusion matrices *(Figures 3 and 4)* show that they each made two mistakes. One running sample was predicted as walking and one walking sample was predicted as taking stairs.

Table 2: Classification Report of k-nearest Neighbours

	precision	recall	f1-score	support
running	1.0	0.974	0.987	38.0
sitting	1.0	1.0	1.0	119.0
stairs	0.983	1.0	0.991	58.0
walking	0.981	0.981	0.981	54.0
accuracy	0.993	0.993	0.993	0.993
macro avg	0.991	0.989	0.99	269.0
weighted avg	0.993	0.993	0.993	269.0

Figure 3: Confusion Matrix of k-nearest Neigbours

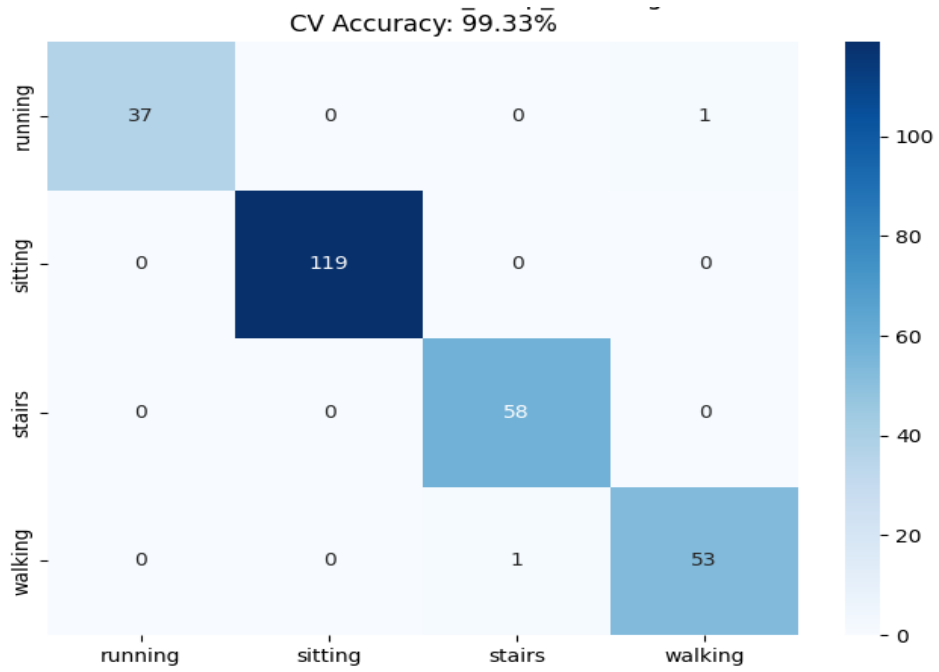


This means the recall for running is slightly lower at 0.97 because the model missed one sample. The precision for walking is also slightly lower at 0.98 because the model incorrectly labeled a running sample as walking. Even with these small errors the F1-scores stay above 0.98 which is an very good result. The confusion matrices are very helpful to interpret these numbers because they show exactly where the models have difficulties. Most of the very small mistakes happened between walking and taking stairs or between running and walking. This is logical because these movements are physically related and they use similar muscle groups. If I move my legs very fast while walking it can look a bit like running to the sensors. Also the motion of the legs when going up a stair is similar to a normal walking step but with a different intensity. Overall the fact that the precision, recall, and F1-score are so high for all activities proves that my thirty-six statistical features are very effective at distinguishing between these four everyday movements.

Table 3: Classification Report of MLP Deep Learning

	precision	recall	f1-score	support
running	1.0	0.974	0.987	38.0
sitting	1.0	1.0	1.0	119.0
stairs	0.983	1.0	0.991	58.0
walking	0.981	0.981	0.981	54.0
accuracy	0.993	0.993	0.993	0.993
macro avg	0.991	0.989	0.99	269.0
weighted avg	0.993	0.993	0.993	269.0

Figure 4: Confusion Matrix of MLP Deep Learning

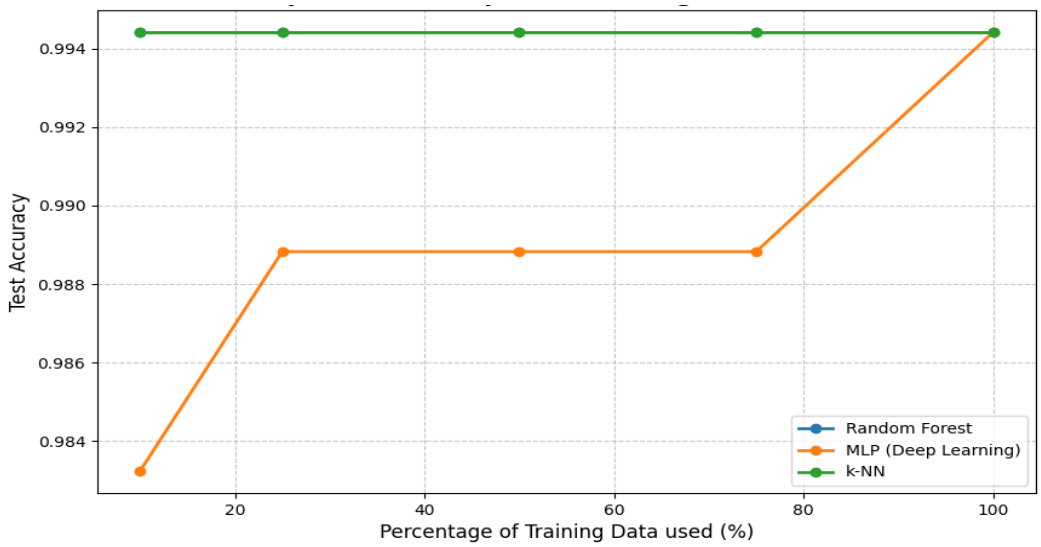


Experiments

In the following experiments I used the random forest model, as it performed best.

For my first experiment, I trained the Random Forest, k-NN, and MLP models using varying percentages of the available training data ranging from 10% to 100%. A fixed test set was used throughout to ensure a fair comparison. As seen in *Figure 5*, Random Forest and k-NN remain remarkably stable even with very little data (Random Forest is behind the green line). In contrast, the MLP shows a clear learning curve, starting at approximately 98.3% accuracy and requiring the full dataset to match the performance of the simpler models. This suggests that for small-scale sensor projects, classic ML is often more efficient than deep learning.

Figure 5: Impact of Data Training Amount



For the second experiment, I used Principal Component Analysis (PCA) to reduce the 36-dimensional feature space into a 2D projection. *Figure 6* shows that PC1 explains 70.2% of the variance, effectively separating "Running" (far right) from "Sitting" (far left). The overlapping clusters for "Walking" and "Stairs" in the center explain the minor confusion noted in the classification reports, proving that these two activities are the most physically similar in the feature space.

In the third experiment, I compared model performance using three configurations: only accelerometer features, only gyroscope features, and the full fusion of both. This experiment addresses hardware efficiency. The results show that the Accelerometer Only configuration is incredibly powerful, achieving 99.78% accuracy, which is identical to the full fusion model. The Gyroscope Only model is slightly lower at 99.55%. This is interesting for mobile development because it suggests a device could potentially turn off the gyroscope to save battery while still maintaining near-perfect activity recognition.

Moreover, I categorized the features into three "Computational Load" levels: Low CPU (the 6 means), Medium CPU (means and standard deviations), and High CPU (all 36 statistical features). Result shows a "sweet spot": moving from Low to Medium CPU (just 12 features) increases accuracy from 96.98% to a staggering 99.66%. However, adding the remaining 24 features for "High CPU" only provides a marginal gain of 0.12%. This suggests that for a real-world app, the "Medium" configuration is the most logical choice for balancing precision and battery life.

Figure 6: PCA - Feature Space Projection

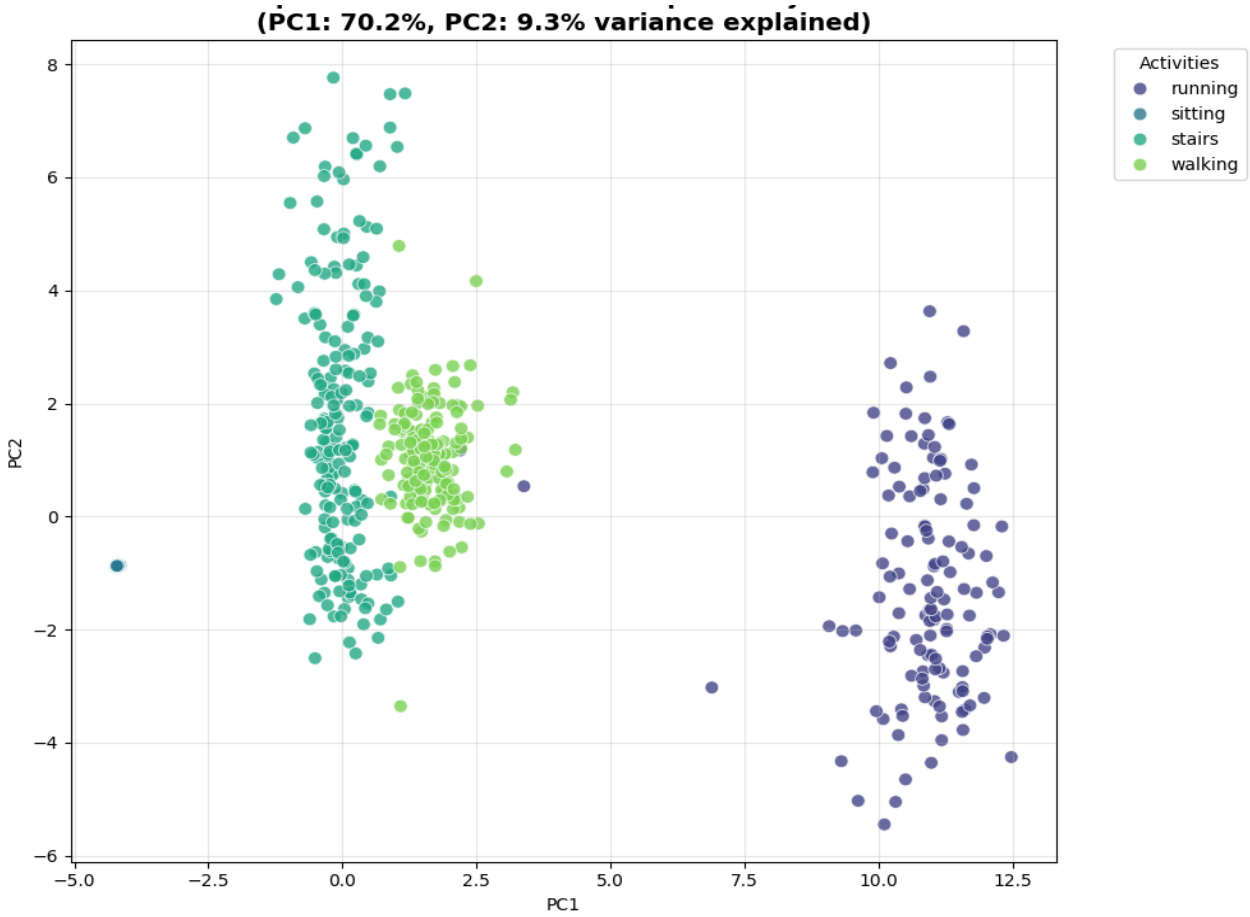


Figure 7: Sensor Importance

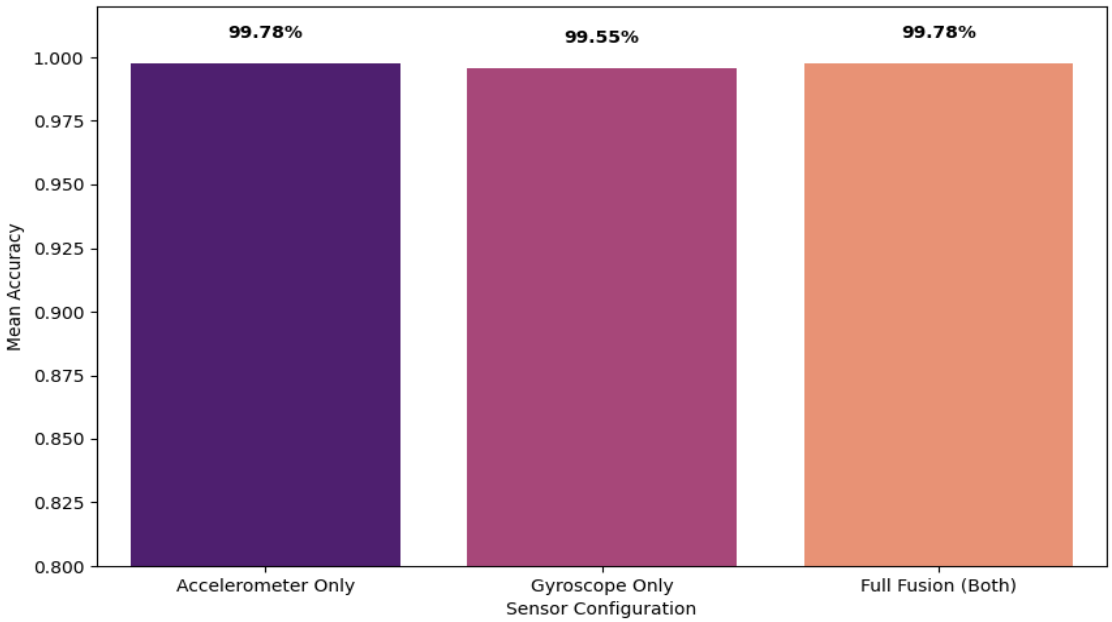
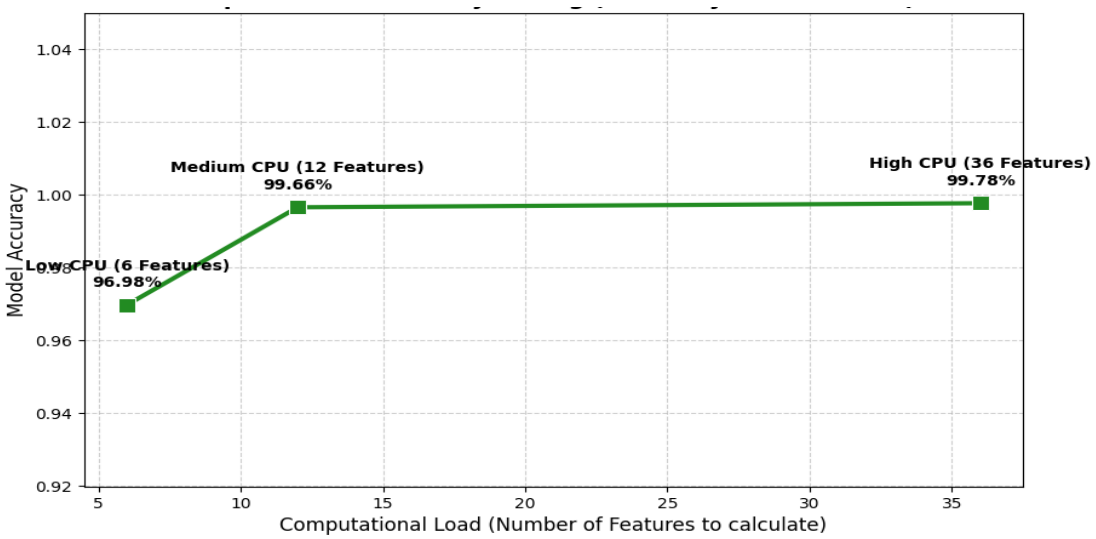


Figure 8: Feature Reduction



Discussion

The model performance, exceeding 99% accuracy, aligns with expectations given the distinct physical nature of the activities. As confirmed by the PCA results, sitting and running have unique sensor signatures—one characterized by stasis and the other by high-intensity periodic impulses—that are mathematically easy to separate. In contrast, the slight confusion between walking and climbing stairs is logical due to their shared rhythmic leg motions and similar vertical acceleration profiles. These results were heavily influenced by the transition from raw data to 36 statistical features, such as Root Mean Square (RMS) and Standard Deviation, which provided a clean, stabilized representation for the models and successfully mitigated transient sensor noise. Experiment 1 confirmed that the MLP is significantly more "data-hungry" than the Random Forest, showing that while neural networks can eventually reach high precision, they require a larger training overhead to optimize their complex internal weights. Meanwhile, the final experiment regarding battery efficiency proved that a reduced configuration of just 12 core features offers the most pragmatic balance between high precision and smartphone battery life, as the marginal 0.12% gain from the full feature set likely does not justify the additional CPU cycles in a real-world mobile environment. Although the self-collected nature of the data ensured a high signal-to-noise ratio and consistent phone orientation, future research should focus on cross-user generalization to account for different gait patterns, varying phone placements, and the detection of complex transitional states like the momentary shift from sitting to standing

Appendix

I used four different code files for my project:

File: merge_clean.py

```
import pandas as pd

import numpy as np

import os

import glob


SAMPLE_RATE   = 50

TRIM_SECONDS  = 15

TRIM_ROWS     = SAMPLE_RATE * TRIM_SECONDS


RAW_FOLDER        = "data/raw"

PROCESSED_FOLDER  = "data/processed"

os.makedirs(PROCESSED_FOLDER, exist_ok=True)


ACTIVITIES = ["walking", "running", "sitting", "stairs", "standing"]


#helper


def find_file(activity, sensor_name):


    base_path = os.path.join(os.getcwd(), RAW_FOLDER)

    print(f"  [DEBUG] Searching in: {base_path}")
```

```
    if not os.path.exists(base_path):
        print(f"  [DEBUG] Directory DOES NOT EXIST: {base_path}")
        return None

    all_files = os.listdir(base_path)
    for f in all_files:
            if activity.lower() in f.lower() and sensor_name.lower() in f.lower() and
f.endswith('.csv'):
            full_path = os.path.join(base_path, f)
            print(f"  [DEBUG] Found file: {f}")
            return full_path

    return None

def load_csv(filepath):
    if filepath is None or not os.path.exists(filepath):
        return None
    df = pd.read_csv(filepath, sep=',')
    if len(df.columns) == 1:
        df = pd.read_csv(filepath, sep='\t')
    print(f"  Loaded: {os.path.basename(filepath)} ({len(df)} rows)")
    return df

def standardize_columns(df, sensor_type):
    df.columns = df.columns.str.strip()
```

```
    time_col = [c for c in df.columns if 'time' in c.lower()]
    if time_col:
        df = df.rename(columns={time_col[0]: 'time'})

    prefix = 'acc' if 'acc' in sensor_type.lower() else 'gyro'
    for col in df.columns:
        if col == 'time': continue
        if 'x' in col.lower(): df = df.rename(columns={col: f'{prefix}_x'})
        elif 'y' in col.lower(): df = df.rename(columns={col: f'{prefix}_y'})
        elif 'z' in col.lower(): df = df.rename(columns={col: f'{prefix}_z'})
    return df

def trim_recording(df):
    if len(df) <= 2 * TRIM_ROWS:
        print("  [WARNING] File too short to trim!")
        return df
    df = df.iloc[TRIM_ROWS:-TRIM_ROWS].reset_index(drop=True)
    return df

def merge_acc_gyro(acc_df, gyro_df):
    acc_df  = acc_df.sort_values('time').reset_index(drop=True)
    gyro_df = gyro_df.sort_values('time').reset_index(drop=True)
    merged = pd.merge_asof(acc_df, gyro_df, on='time', direction='nearest', tolerance=0.05)
    merged = merged.dropna().reset_index(drop=True)
    print(f"  Merged successfully: {len(merged)} rows")
```

```
    return merged


def process_activity(activity):
    print(f"\n{'='*50}\n  {activity.upper()}\n{'='*50}")

    # Search for files using flexible patterns
    acc_path  = find_file(activity, "Linear")
    gyro_path = find_file(activity, "Gyroscope")

    if not acc_path or not gyro_path:
        print(f"  [SKIPPING] Missing files for {activity}")
        return None

    acc_df  = load_csv(acc_path)
    gyro_df = load_csv(gyro_path)

    acc_df  = standardize_columns(acc_df, 'acc')
    gyro_df = standardize_columns(gyro_df, 'gyro')

    acc_df  = trim_recording(acc_df)
    gyro_df = trim_recording(gyro_df)

    merged = merge_acc_gyro(acc_df, gyro_df)
    merged['activity'] = activity
```

```
    out_path = os.path.join(PROCESSED_FOLDER, f"{activity}_merged.csv")
    merged.to_csv(out_path, index=False)
    return merged

if __name__ == "__main__":
    all_data = []
    for activity in ACTIVITIES:
        result = process_activity(activity)
        if result is not None:
            all_data.append(result)

    if len(all_data) > 0:
        combined = pd.concat(all_data, ignore_index=True)
        combined.to_csv(os.path.join(PROCESSED_FOLDER, "all_activities.csv"),
index=False)
        print(f"\nDONE! Combined file saved in {PROCESSED_FOLDER}")
```

File: extract_features.py

```
import pandas as pd

import numpy as np

import os


#settings


WINDOW_SIZE_SEC = 2

OVERLAP_PCT     = 0.5

SAMPLING_RATE   = 50


PROCESSED_FOLDER = "data/processed"

FEATURES_OUTPUT  = os.path.join(PROCESSED_FOLDER, "final_features.csv")


#create features


def extract_features_from_window(window, activity_label):

    """

    Calculates statistical features for a single 2-second window.

    These features represent the 'fingerprint' of the movement.

    """

    features = {}

    #only sensor

    sensor_cols = [c for c in window.columns if c not in ['time', 'activity']]
```

```
    for col in sensor_cols:

        features[f'{col}_mean'] = window[col].mean()
        features[f'{col}_std']  = window[col].std()
        features[f'{col}_max']  = window[col].max()
        features[f'{col}_min']  = window[col].min()

        #Signal Intensity (RMS)
        features[f'{col}_rms']  = np.sqrt(np.mean(window[col]**2))

        features[f'{col}_median'] = window[col].median()

    features['label'] = activity_label
    return features

def process_all_activities():

    all_feature_rows = []

    files = [f for f in os.listdir(PROCESSED_FOLDER) if f.endswith('_merged.csv')]

    print(f"Found {len(files)} activity files. Starting windowing...")

    window_step = int(SAMPLING_RATE * WINDOW_SIZE_SEC)
```

```
step_size = int(window_step * (1 - OVERLAP_PCT))


for file_name in files:
    file_path = os.path.join(PROCESSED_FOLDER, file_name)
    df = pd.read_csv(file_path)
    activity = df['activity'].iloc[0]


    print(f"  Processing {activity}...")


    count = 0


    for start in range(0, len(df) - window_step, step_size):
        window = df.iloc[start : start + window_step]



        feature_row = extract_features_from_window(window, activity)
        all_feature_rows.append(feature_row)
        count += 1


    print(f"    -> Generated {count} windows for {activity}")


#final dataframe
final_df = pd.DataFrame(all_feature_rows)
final_df.to_csv(FEATURES_OUTPUT, index=False)
```

```
    print("\n" + "="*30)
    print(f"SUCCESS: {FEATURES_OUTPUT} created.")
    print(f"Total dataset size: {final_df.shape}")
    print("Class distribution:")
    print(final_df['label'].value_counts())
    print("="*30)

if __name__ == "__main__":
    process_all_activities()
```

File: train_evaluate.py

```
import pandas as pd

import numpy as np

import os

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold

from sklearn.preprocessing import StandardScaler

from sklearn.ensemble import RandomForestClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score


def save_table_as_image(df, filename, title):

    fig, ax = plt.subplots(figsize=(10, 4))

    ax.axis('off')

    ax.set_title(title, fontsize=16, fontweight='bold', pad=20)


    #table

    table = ax.table(cellText=df.values,

                     colLabels=df.columns,

                         rowLabels=df.index if df.index.name != None or not isinstance(df.index,
pd.RangeIndex) else None,

                     cellLoc='center',
```

```
                    loc='center')


    table.auto_set_font_size(False)
    table.set_fontsize(12)
    table.scale(1.2, 1.5)


    for (row, col), cell in table.get_celld().items():
        if row == 0:
            cell.set_text_props(weight='bold')

    plt.tight_layout()
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    plt.close()


INPUT_FILE = "data/processed/final_features.csv"
RESULTS_DIR = "results"
os.makedirs(RESULTS_DIR, exist_ok=True)

#load data
df = pd.read_csv(INPUT_FILE)
X = df.drop('label', axis=1)
y = df['label']
```

```
#scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#model setup
models = {
    "Random_Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "k-Nearest_Neighbors": KNeighborsClassifier(n_neighbors=5),
    "MLP_Deep_Learning": MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=1000, random_state=42)
}

#validation
cv_results = {}
final_report = []

#stratified
skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

print(f"{'Model':<20} | {'CV Mean Acc':<12} | {'Std Dev':<8}")
print("-" * 45)

for name, model in models.items():
```

```
    current_X = X_scaled if name != "Random_Forest" else X



    scores = cross_val_score(model, current_X, y, cv=skf)


    mean_acc = scores.mean()
    std_acc = scores.std()
    cv_results[name] = mean_acc


    print(f"{name:<20} | {mean_acc:.4f}      | {std_acc:.4f}")


    #Train-Test Split


    X_train, X_test, y_train, y_test = train_test_split(current_X, y, test_size=0.3, random_state=42, stratify=y)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)


    #report
    with open(f"{RESULTS_DIR}/report_{name}.txt", "w") as f:
        f.write(f"Model: {name}\n")
        f.write(f"Cross-Validation Mean Accuracy: {mean_acc:.4f} (+/- {std_acc:.4f})\n\n")
        f.write(classification_report(y_test, y_pred))


    #Confusion Matrix
```

```
    plt.figure(figsize=(8,6))
    cm = confusion_matrix(y_test, y_pred)
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=model.classes_, yticklabels=model.classes_)
    plt.title(f"Confusion Matrix: {name}\nCV Accuracy: {mean_acc:.2%}")
    plt.savefig(f"{RESULTS_DIR}/cm_{name}.png")
    plt.close()

    report_dict = classification_report(y_test, y_pred, output_dict=True)
    report_df = pd.DataFrame(report_dict).transpose().round(3)
            save_table_as_image(report_df, f"{RESULTS_DIR}/table_img_{name}.png", f"Classification Report: {name}")
```

```
#plot

plt.figure(figsize=(10, 8))

bars = plt.bar(cv_results.keys(), cv_results.values(), color=['#4C72B0', '#55A868', '#C44E52'])
plt.ylabel('Mean CV Accuracy', fontsize=14)
plt.title('10-Fold Cross-Validation Performance', fontsize=16, fontweight='bold')
plt.ylim(0, 1.1)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
```

```
for i, v in enumerate(cv_results.values()):
    plt.text(i, v + 0.02, f"{v:.2%}", ha='center', fontsize=13, fontweight='bold')


plt.tight_layout() # Ensures nothing is cut off
plt.savefig(f"{RESULTS_DIR}/cv_accuracy_comparison.png")
```

File:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import os


from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.ensemble import RandomForestClassifier

from sklearn.neural_network import MLPClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.metrics import accuracy_score, f1_score


#Settings

INPUT_FILE = "data/processed/final_features.csv"

OUTPUT_DIR = "results/experiments"

os.makedirs(OUTPUT_DIR, exist_ok=True)


#Load Data

df = pd.read_csv(INPUT_FILE)

X = df.drop('label', axis=1)

y = df['label']
```

```
#Global Scaling
scaler = StandardScaler()
X_scaled_values = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled_values, columns=X.columns)


#1. Learning Curve
def run_data_hunger_experiment():


    percentages = [0.1, 0.25, 0.5, 0.75, 1.0]
    results = {"Random Forest": [], "MLP (Deep Learning)": [], "k-NN": []}


    X_train_full, X_test, y_train_full, y_test = train_test_split(
        X_scaled, y, test_size=0.2, random_state=42, stratify=y
    )


    for p in percentages:
        if p < 1.0:
            X_sub, _, y_sub, _ = train_test_split(
                X_train_full, y_train_full, train_size=p, random_state=42, stratify=y_train_full
            )
        else:
            X_sub, y_sub = X_train_full, y_train_full


        models = {
            "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
```

```
        "MLP (Deep Learning)": MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=1000, random_state=42),
        "k-NN": KNeighborsClassifier(n_neighbors=5)
    }

    for name, model in models.items():
        model.fit(X_sub, y_sub)
        results[name].append(accuracy_score(y_test, model.predict(X_test)))

plt.figure(figsize=(10, 6))
for model_name, accs in results.items():
    plt.plot(np.array(percentages)*100, accs, marker='o', linewidth=2, label=model_name)

plt.title("Experiment 1: Impact of Training Data Amount", fontsize=14, fontweight='bold')
plt.xlabel("Percentage of Training Data used (%)", fontsize=12)
plt.ylabel("Test Accuracy", fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.savefig(f"{OUTPUT_DIR}/01_data_hunger.png")
plt.close()

#PCA Vizualisation

def run_pca_experiment():
    print("Running Experiment 2: PCA Visualization...")
```

```
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X_scaled)

    pca_df = pd.DataFrame(data=X_pca, columns=['PC1', 'PC2'])
    pca_df['label'] = y

    plt.figure(figsize=(10, 8))
        sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='label', palette='viridis', s=60, alpha=0.8)

    var_exp = pca.explained_variance_ratio_
     plt.title(f"Experiment 2: PCA - Feature Space Projection\n(PC1: {var_exp[0]:.1%}, PC2: {var_exp[1]:.1%} variance explained)", fontsize=14, fontweight='bold')
    plt.legend(title="Activities", bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(f"{OUTPUT_DIR}/02_pca_projection.png")
    plt.close()

#Sensor
def run_sensor_ablation_experiment():
    print("Running Experiment 3: Sensor Importance...")

    acc_cols = [c for c in X.columns if 'acc' in c.lower()]
```

```
gyro_cols = [c for c in X.columns if 'gyro' in c.lower()]

sensor_sets = {
    "Accelerometer Only": X_scaled[acc_cols],
    "Gyroscope Only": X_scaled[gyro_cols],
    "Full Fusion (Both)": X_scaled
}

ablation_results = []

for label, data in sensor_sets.items():
    #Use RF
    rf = RandomForestClassifier(n_estimators=100, random_state=42)
    scores = cross_val_score(rf, data, y, cv=5)
        ablation_results.append({"Sensor Configuration": label, "Mean Accuracy": scores.mean()})

res_df = pd.DataFrame(ablation_results)

plt.figure(figsize=(10, 6))
    sns.barplot(data=res_df, x="Sensor Configuration", y="Mean Accuracy", palette="magma")
plt.ylim(0.8, 1.02)
plt.title("Experiment 3: Sensor Importance Study", fontsize=14, fontweight='bold')
for i, v in enumerate(res_df["Mean Accuracy"]):
```

```
        plt.text(i, v + 0.01, f"{v:.2%}", ha='center', fontweight='bold')

    plt.savefig(f"{OUTPUT_DIR}/03_sensor_ablation.png")
    plt.close()

#Class Imbalance
def run_imbalance_experiment():
    print("Running Experiment 4: Handling Class Imbalance...")

        X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42, stratify=y)

    rf_standard = RandomForestClassifier(n_estimators=100, random_state=42)
        rf_balanced = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')

    rf_standard.fit(X_train, y_train)
    rf_balanced.fit(X_train, y_train)

    f1_std = f1_score(y_test, rf_standard.predict(X_test), average='weighted')
    f1_bal = f1_score(y_test, rf_balanced.predict(X_test), average='weighted')

    imb_res = pd.DataFrame({
        "Method": ["Standard Random Forest", "Balanced Random Forest"],
        "Weighted F1-Score": [f1_std, f1_bal]
```

```
    })

    plt.figure(figsize=(8, 6))
    sns.barplot(data=imb_res, x="Method", y="Weighted F1-Score", palette="coolwarm")
    plt.ylim(0.95, 1.01)
    plt.title("Experiment 4: Impact of Class Weighting (Imbalance)", fontsize=14, fontweight='bold')
    for i, v in enumerate(imb_res["Weighted F1-Score"]):
        plt.text(i, v + 0.002, f"{v:.4f}", ha='center', fontweight='bold')

    plt.savefig(f"{OUTPUT_DIR}/04_imbalance_handling.png")
    plt.close()

#feature reduction
def run_battery_saving_experiment():
    print("Running Experiment 5: Battery Saving vs. Accuracy...")

    #different subsets (6, 12, 36)

    feature_sets = {
        "Low CPU (6 Features)": [c for c in X.columns if 'mean' in c.lower()],
        "Medium CPU (12 Features)": [c for c in X.columns if 'mean' in c.lower() or 'std' in c.lower()],
        "High CPU (36 Features)": X.columns.tolist()
    }
```

```
battery_results = []

for label, cols in feature_sets.items():
    data = X_scaled[cols]

    rf = RandomForestClassifier(n_estimators=100, random_state=42)
    scores = cross_val_score(rf, data, y, cv=5)
    battery_results.append({
        "Complexity Level": label,
        "Num Features": len(cols),
        "Mean Accuracy": scores.mean()
    })

res_df = pd.DataFrame(battery_results)

# Plotting
plt.figure(figsize=(10, 6))
sns.lineplot(data=res_df, x="Num Features", y="Mean Accuracy", marker='s', markersize=10, linewidth=3, color='forestgreen')

plt.title("Experiment 5: Battery Saving (Accuracy vs. CPU Load)", fontsize=14, fontweight='bold')
plt.xlabel("Computational Load (Number of Features to calculate)", fontsize=12)
plt.ylabel("Model Accuracy", fontsize=12)
```

```
    plt.grid(True, linestyle='--', alpha=0.6)


    # Annotate points
    for i, row in res_df.iterrows():
        plt.text(row["Num Features"], row["Mean Accuracy"] + 0.005,
                 f"{row['Complexity Level']}\n{row['Mean Accuracy']:.2%}",
                 ha='center', fontweight='bold', fontsize=10)


    plt.ylim(min(res_df["Mean Accuracy"]) - 0.05, 1.05)
    plt.savefig(f"{OUTPUT_DIR}/05_battery_saving_study.png")
    plt.close()
    print(f"  -> Plot saved: {OUTPUT_DIR}/05_battery_saving_study.png")


if __name__ == "__main__":
    run_data_hunger_experiment()
    run_pca_experiment()
    run_sensor_ablation_experiment()
    run_imbalance_experiment()
    run_battery_saving_experiment()
    print("\nSuccess! All 4 experiments are saved in 'results/experiments/'.")
```